

Technical Recruitment Tasks

MLSA SRM

Microsoft Learn Student Ambassadors, SRM Institute of Science and Technology

Part 1 Welcome

If you're reading this, you've either picked up a technical task from the recruitment portal or you're about to. This guide covers every task in the Technical domain: what to build, what resources you'll need, how second years extend the same task a bit further, and exactly how to submit your work.

Read the section for your task fully before you start. Most of the questions people ask in the WhatsApp group are already answered here.

HEADLINE

One rule applies to every task on this list: first years and second years get the exact same base task. Second years then have one additional section to build on top. Nobody gets an easier version.

Part 2 What We're Actually Looking For

We'd rather see an honest, partial attempt with a clear writeup of what you tried and where you got stuck than a polished submission you can't explain. Trying matters here, genuinely. If you spend three hours wrestling with WebSockets and get half of it working, tell us that in your README. We read those.

That said, "I tried" is a starting point, not a finish line. We're still looking for working code, real understanding, and a submission you can talk through confidently. Effort earns you the benefit of the doubt when something is rough around the edges. It doesn't replace the thing itself.

The single best way to show us you understand your own code is the video walkthrough described in Part 3. It's short, it's low-pressure, and it's the part almost nobody fakes well by accident.

Part 3 How to Submit

Every task submission needs three things, unless a task's own section says otherwise:

1. **A public GitHub repository link.** It needs to be public so we can actually open and review it. Commit history matters a little: a single commit dumping the whole project reads very differently from a history that shows you working through it.
2. **A README** covering how to run the project locally, what you built, and anything you didn't finish or would do differently with more time. Two or three honest paragraphs is plenty.
3. **A 2 to 3 minute video walkthrough.** Screen record yourself running the project and talking through one decision you made and why. Any unlisted YouTube link, Loom link, or Google Drive link works. This is the one part of the submission we ask you not to skip: it's how we tell "built and understood this" apart from "generated and pasted this."

Some tasks ask for one extra thing on top of these three (a deployed link, a notebook, a short benchmark writeup). That's called out in the task's own section below.

Submit through the recruitment portal against the task you picked. If you're a second year, submit the base task and the second-year section together as one submission, one repo, one video. Don't submit them separately.

Part 4 Frontend Tasks

4.1 Live-Sync Mini App

Base task. Build a small app where multiple open browser tabs see each other's actions in real time: a shared sticky-note board, live cursor positions, or a shared poll that updates as votes come in. Use React for the interface and either WebSockets or

Server-Sent Events for the real-time part. Two tabs open side by side should be enough to demo it.

What you'll need. Node 18 or newer. A WebSocket server is easiest with a small Express or Fastify backend and the `ws` library, or Socket.IO if you'd rather have reconnection handling built in. If you go the SSE route, you only need a single HTTP endpoint that stays open and streams events, no extra library required.

Resources.

- WebSocket API on MDN: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- Server-Sent Events on MDN: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- Socket.IO docs: <https://socket.io/docs/v4/>

Second-year build-on. Add optimistic UI updates, and handle two tabs editing the same note or cursor position at the same moment without one silently overwriting the other. Then make the state survive a refresh: closing a tab and reopening it should restore where things were, not reset to empty.

Submission. The standard three items from Part 3. Include, in your README, a one-line note on how you resolved the simultaneous-edit case.

4.2 Fix This Broken UI

Base task. We'll hand you a small React app with four bugs planted in it on purpose: a stale closure that shows the wrong value, a missing list key causing weird re-render behavior, a layout bug that only appears on narrow screens, and an accessibility issue a screen reader would catch. Find all four, fix them, and add a one-line comment at each fix explaining what was actually wrong.

This is a debugging task, not a rebuild. Please don't rewrite the app from scratch, we specifically want to see how you diagnose someone else's code.

Starter repo. <https://github.com/MLSA-SRM/recruit-task-fix-broken-ui>

What you'll need. Node 18 or newer, and a browser with dev tools (React DevTools is genuinely useful here). For the accessibility bug, VoiceOver on macOS or NVDA on Windows will catch it directly, but you can also spot it with the axe DevTools browser extension if you don't want to run a full screen reader.

Resources.

- React docs, particularly the section on state and closures:
<https://react.dev/learn>
- ARIA Authoring Practices Guide: <https://www.w3.org/WAI/ARIA/apg/>
- axe DevTools: <https://www.deque.com/axe/devtools/>

Second-year build-on. After the four fixes, add one small new feature to the same app (your choice, keep it small). The new feature has to be keyboard-accessible on its own, and covered by at least one automated test using React Testing Library.

Submission. The standard three items. In your video, walk through each of the four bugs and point at the actual line that was wrong, not just the symptom.

4.3 Accessible Custom Component

Base task. Build one non-trivial UI component completely from scratch, no component library: a multi-select dropdown, a virtualized long list, or a drag-to-reorder list. It needs full keyboard navigation and correct focus management, not just mouse support.

What you'll need. Just React and Node 18 or newer. Deliberately skip Radix, MUI, and similar libraries for this one, the point is building the primitive yourself.

Resources.

- ARIA Authoring Practices Guide (has worked examples for exactly this):
<https://www.w3.org/WAI/ARIA/apg/>
- MDN's keyboard-accessible guide: https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Keyboard-navigable_JavaScript

Second-year build-on. Make the component work correctly with a screen reader, not just a keyboard. Then support both a controlled and an uncontrolled usage mode, the way real component libraries do (compare how a native `<input>` supports both `value` and `defaultValue`).

Submission. The standard three items. A short Storybook page or even a plain demo page showing the component in both usage modes is welcome but not required.

Part 5 Backend Tasks

5.1 Optimize This Slow Endpoint

Base task. We'll hand you a small, working API with one endpoint that's deliberately slow. Profile it, find the actual cause, fix it, and write down the before and after numbers.

Starter repo. <https://github.com/MLSA-SRM/recruit-task-slow-endpoint>

What you'll need. Node 18 or newer, and nothing else. Run it with `npm install` then `npm start` . There's no database to provision, the starter ships with a data layer that simulates realistic query latency. The endpoint already returns a `durationMs` field, so you can measure before and after without any profiling tools.

Resources.

- Explaining SQL query plans (Postgres):
<https://www.postgresql.org/docs/current/using-explain.html>
- The N+1 query problem, explained plainly: <https://planetscale.com/blog/what-is-n-1-query-problem-and-how-to-solve-it>

Second-year build-on. Add caching in front of the now-fixed endpoint (in-memory with a TTL is fine, Redis if you want the practice) and write a short load test showing throughput with and without the cache.

Resources for the extension.

- Redis docs: <https://redis.io/docs/latest/>
- k6 load testing docs: <https://k6.io/docs/>

Submission. The standard three items, plus your before/after numbers (a couple of sentences in the README is enough, no need for a formal report).

5.2 API in an Unfamiliar Framework

Base task. Build a small CRUD API with authentication, but in a framework you haven't used before. If you're comfortable in Node/Express, try FastAPI. If you're a Python person, try Express or a minimal Go HTTP server. The concepts (routing, validation, auth, status codes) are ones you already know, the point is proving you can carry them into new syntax.

What you'll need. Whatever runtime the framework needs (Node 18+, Python 3.10+, or Go 1.21+). Keep auth simple, a signed JWT or even a hardcoded API key check is fine, we're not evaluating your auth architecture here.

Resources.

- FastAPI docs: <https://fastapi.tiangolo.com/>
- Express docs: <https://expressjs.com/>
- Go's net/http package: <https://pkg.go.dev/net/http>

Second-year build-on. Add rate limiting and structured logging (JSON logs with a request ID on every line, not just `console.log`), then containerize the whole thing with a working Dockerfile someone else could `docker run` without reading your code first.

Resources for the extension.

- Docker's getting started guide: <https://docs.docker.com/get-started/>

Submission. The standard three items. Note in your README which framework you picked and one thing about it that surprised you.

5.3 Background Job Queue

Base task. Build a small job queue: something submits jobs (an API endpoint is fine), a worker processes them asynchronously, and failed jobs retry a couple of times before giving up. BullMQ with Redis is the well-trodden path, but a simpler cron-plus-database-polling approach is completely acceptable if you'd rather build it from first principles.

What you'll need. Node 18+ (or Python with Celery if you'd rather), and Redis if you go the BullMQ route (a free Upstash Redis instance works fine, no local install needed).

Resources.

- BullMQ docs: <https://docs.bullmq.io/>
- Upstash Redis (free tier, no credit card): <https://upstash.com/>
- Celery docs (Python alternative): <https://docs.celeryq.dev/en/stable/>

Second-year build-on. Add a dead-letter queue for jobs that fail every retry, and build a tiny status page (even a single unstyled HTML page is fine) showing how many jobs are pending, running, succeeded, and dead.

Submission. The standard three items. In your video, actually trigger a job that fails on purpose and show it land in the dead-letter queue.

Part 6 AI/ML Tasks

6.1 RAG Mini Q&A Bot

Base task. We'll give you a small set of documents. Build a question-answering tool that actually retrieves relevant passages from those specific documents before answering, rather than just forwarding the question straight to an LLM. We want to see the retrieval step, not just a wrapped API call.

Starter documents. <https://github.com/MLSA-SRM/recruit-task-rag-docs>

What you'll need. Python 3.10+, an embeddings model (OpenAI's embeddings API, or a free local model via `sentence-transformers` if you'd rather not use paid credits), and a simple vector store (even an in-memory list with cosine similarity is fine at this scale, you don't need a dedicated vector database).

Resources.

- LangChain's RAG concepts guide:
<https://python.langchain.com/docs/concepts/rag/>
- `sentence-transformers` (free local embeddings): <https://www.sbert.net/>
- A plain-language explanation of embeddings and retrieval:
<https://simonwillison.net/2023/Oct/23/embeddings/>

Second-year build-on. Show which document and passage each answer actually came from (a citation, not just a confident-sounding answer). Then handle the case where the answer genuinely isn't in the documents: the bot should say so rather than making something up.

Submission. The standard three items. Your video should include one question with a real answer and citation, and one question the documents don't cover, to show both behaviors.

6.2 Messy Dataset, Baseline Model

Base task. We'll give you a small, deliberately messy CSV (missing values, a few outliers, inconsistent types in places). Explore it, clean what needs cleaning, build a baseline classifier or regressor, and write up what you found and why you made the choices you made. We're evaluating your reasoning here as much as your model's accuracy.

Starter dataset. <https://github.com/MLSA-SRM/recruit-task-messy-dataset>

What you'll need. Python 3.10+, pandas, and scikit-learn. A Jupyter notebook or Google Colab is the natural format for this one.

Resources.

- pandas docs: <https://pandas.pydata.org/docs/>
- scikit-learn's getting started guide: https://scikit-learn.org/stable/getting_started.html
- Google Colab (free, no setup): <https://colab.research.google.com/>

Second-year build-on. Compare at least two different modeling approaches and justify, specifically for this dataset, which one you'd actually ship. Add a short error analysis: look at what the model gets wrong and describe the pattern, not just the overall accuracy number.

Submission. A GitHub repo or a shared, viewable Colab notebook link both work for this task in place of a traditional repo. The README/writeup and video walkthrough are still required.

Part 7 Getting Unstuck

Stuck is a normal part of every one of these tasks. Ask in the WhatsApp group: <https://chat.whatsapp.com/BfZprjZU1R50RCtCEueR8N?s=qt&p=a&mlu=4>

A few things that make it easier for us to actually help:

- Say what you already tried, not just “it doesn't work.” “I tried X and got error Y” gets a fast answer. “Nothing is working” doesn't.
- Screenshots and error messages, pasted directly, beat descriptions of them.
- There's no such thing as a question that's too basic to ask. The only bad question here is the one you didn't ask and stayed stuck on for two days instead.

HEADLINE

Asking for help isn't a mark against you. Getting stuck, asking a sharp question, and unblocking yourself is a large part of what the job actually looks like.

Part 8 Deadline and Logistics

Deadlines are set per task on the recruitment portal itself, check the task page for the exact date and time. Submit before it closes, late submissions aren't reviewed except in genuinely exceptional circumstances, and those need to be raised with an admin before the deadline, not after.

If you're unsure which task to pick, pick the one that scares you slightly more than the others. That's usually the one worth doing.